

SSE 로 로딩 프로그레스 구현하기

Team Ecode

기현명 김가연 김성빈 이다현 정서윤



✓ FE

- 버그 해결 및 피드백 받은 내용 개선
- 반응형 구현
- CI/CD 구축
- 카카오 로그인 기능 구현

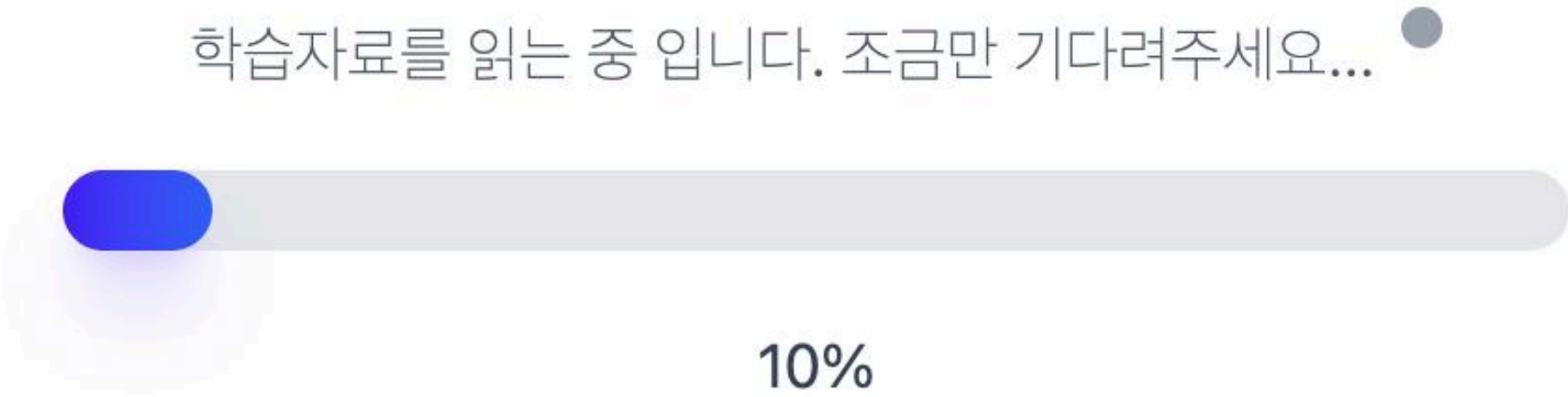
✓ BE

- S3 PRESIGNED URL
- 카카오 로그인 추가
- CICD 구축
- DB 쿼리 최적화

✓ AI

- 문서량 관련 통신 에러 해결
- 채점 최적화
- 버그 수정

로딩 프로그래스바



Long polling vs Web Socket vs SSE

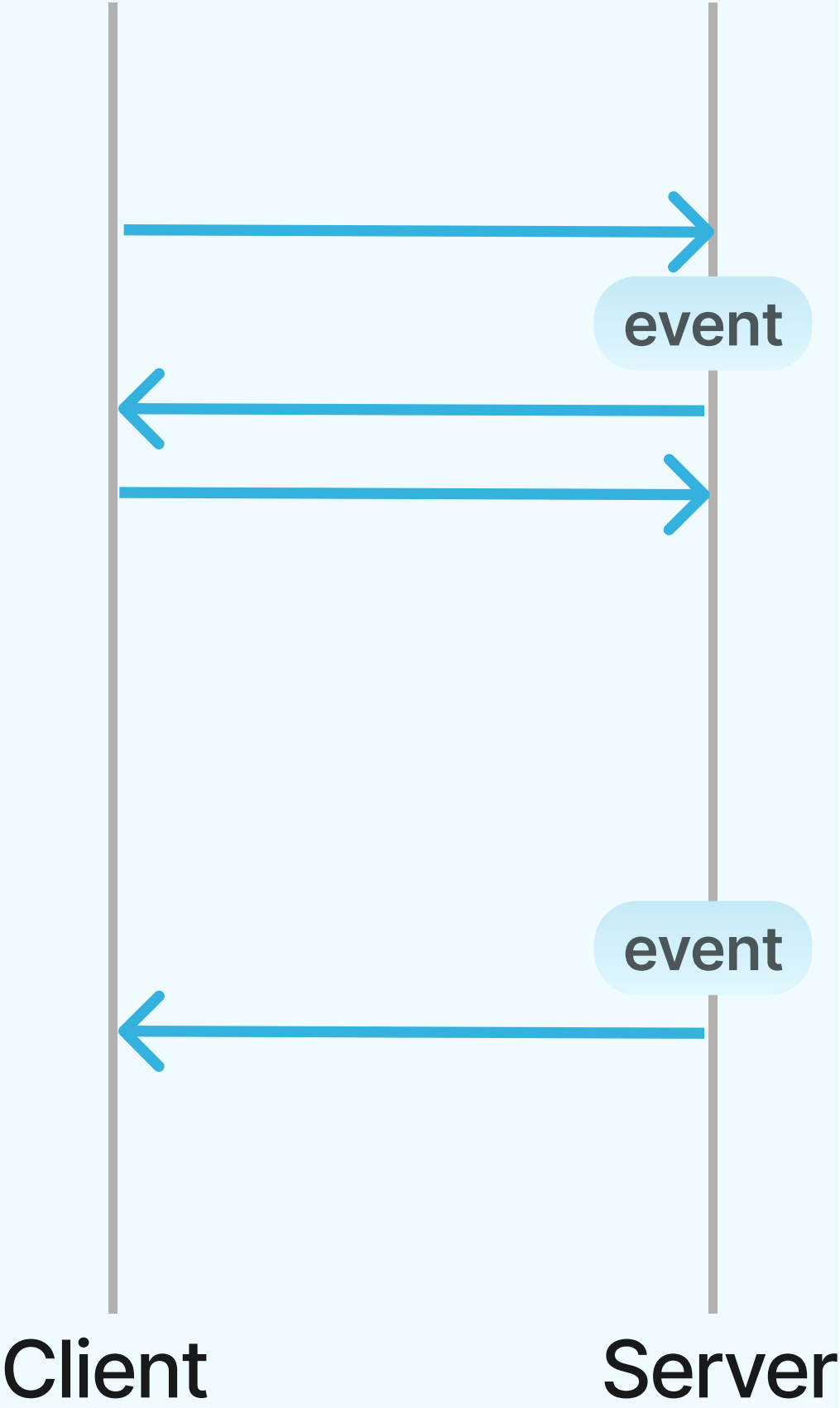
protocol

http

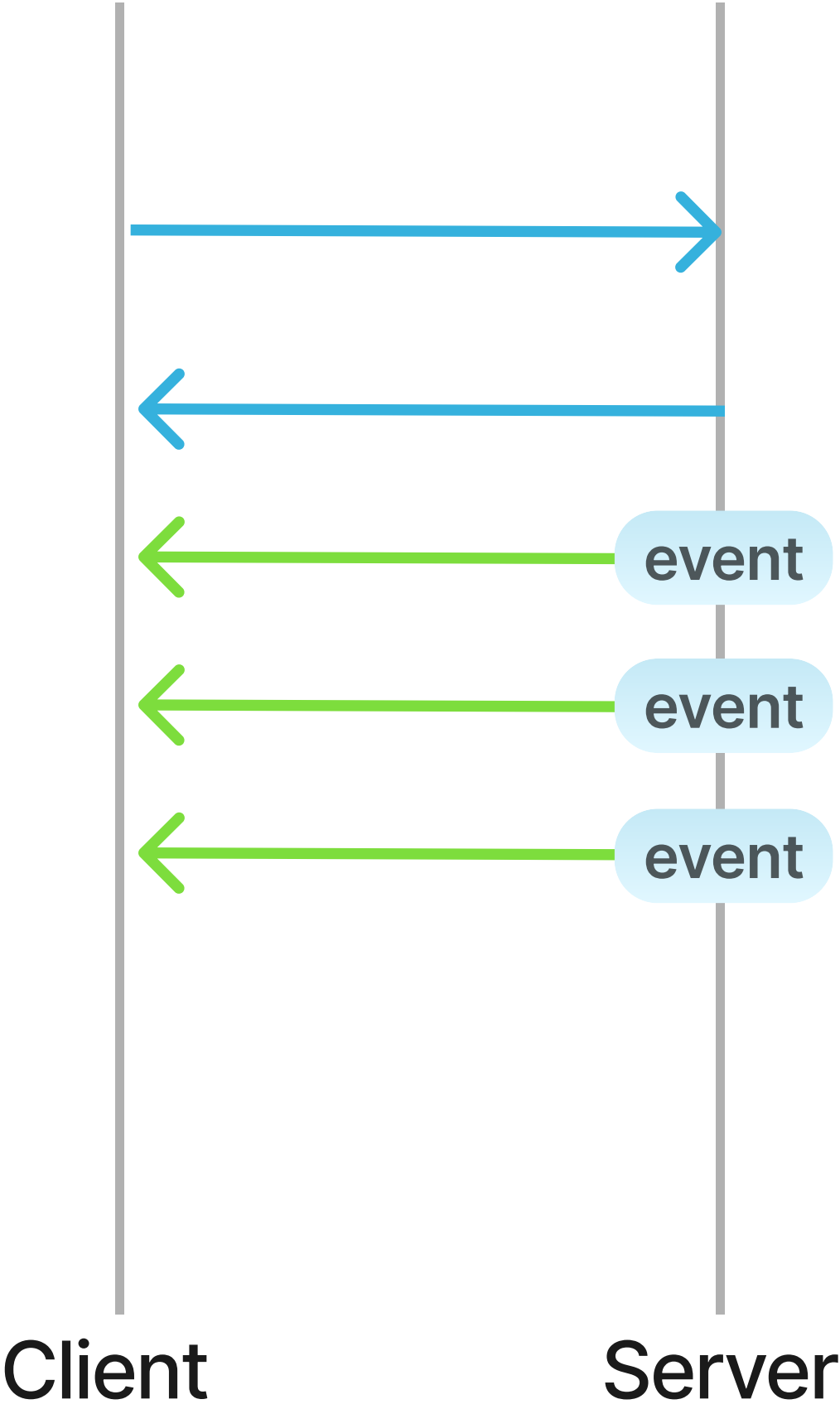
EventSource

WebSocket

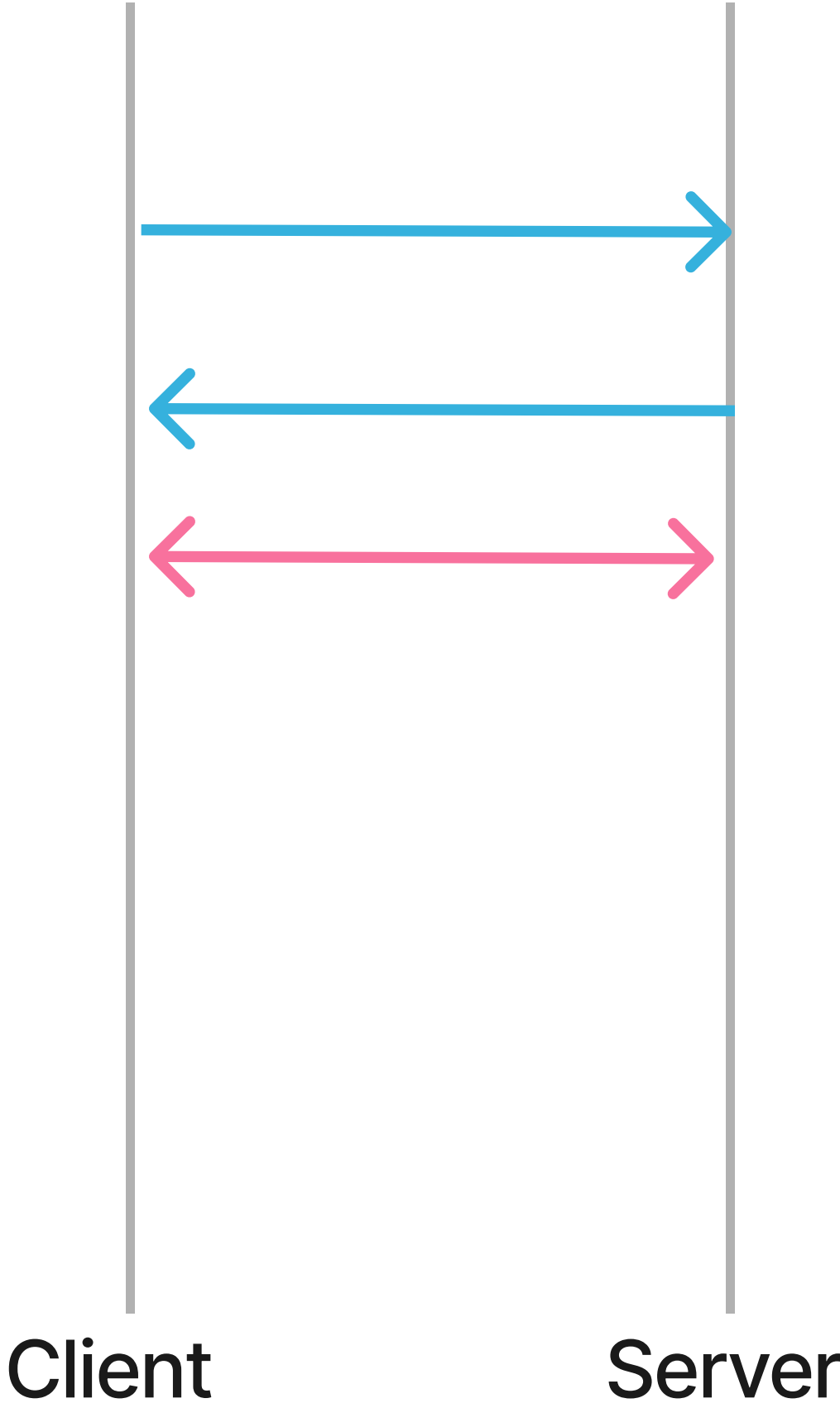
Long polling



SSE



WebSocket



Long polling vs Web Socket vs SSE

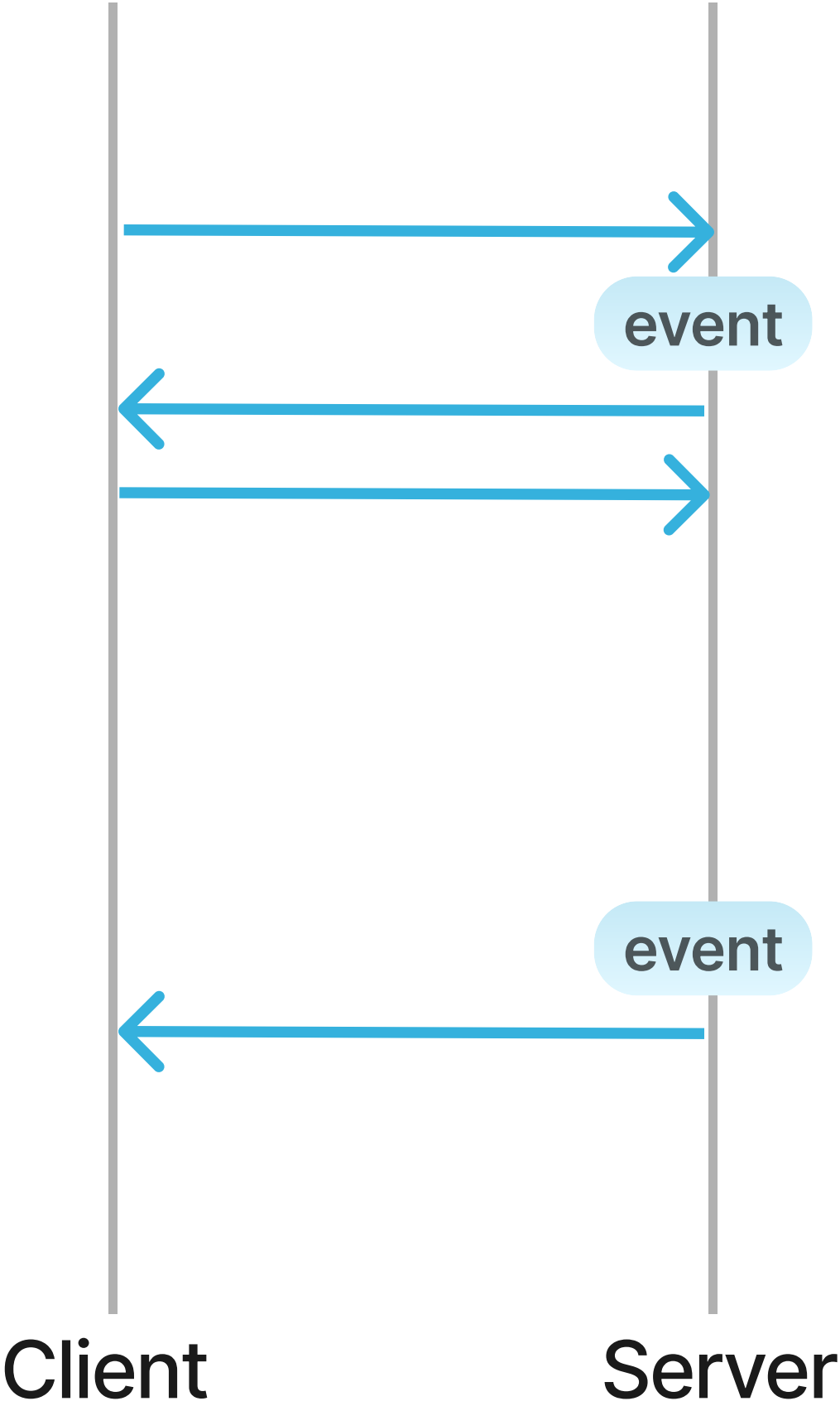
protocol

http

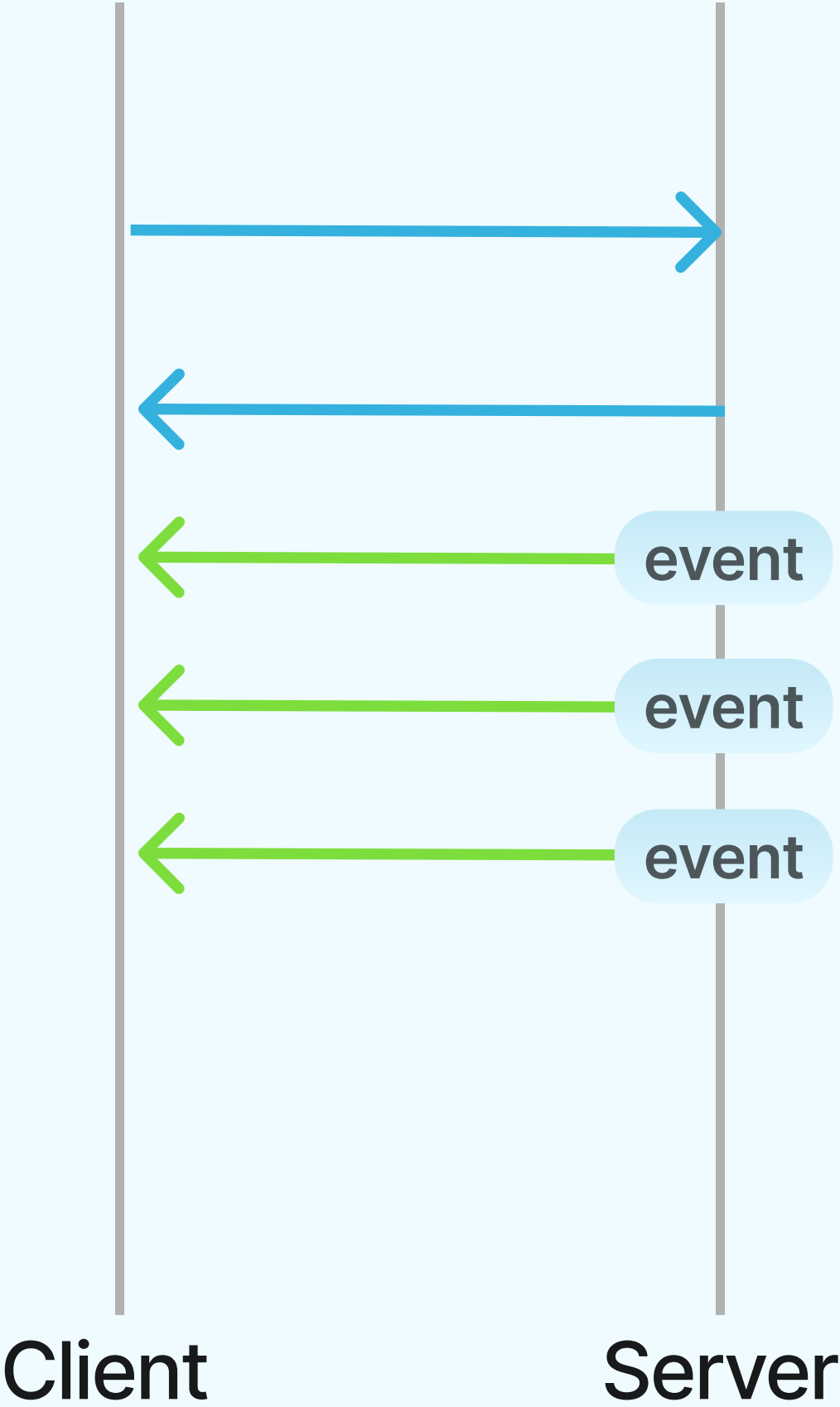
EventSource

WebSocket

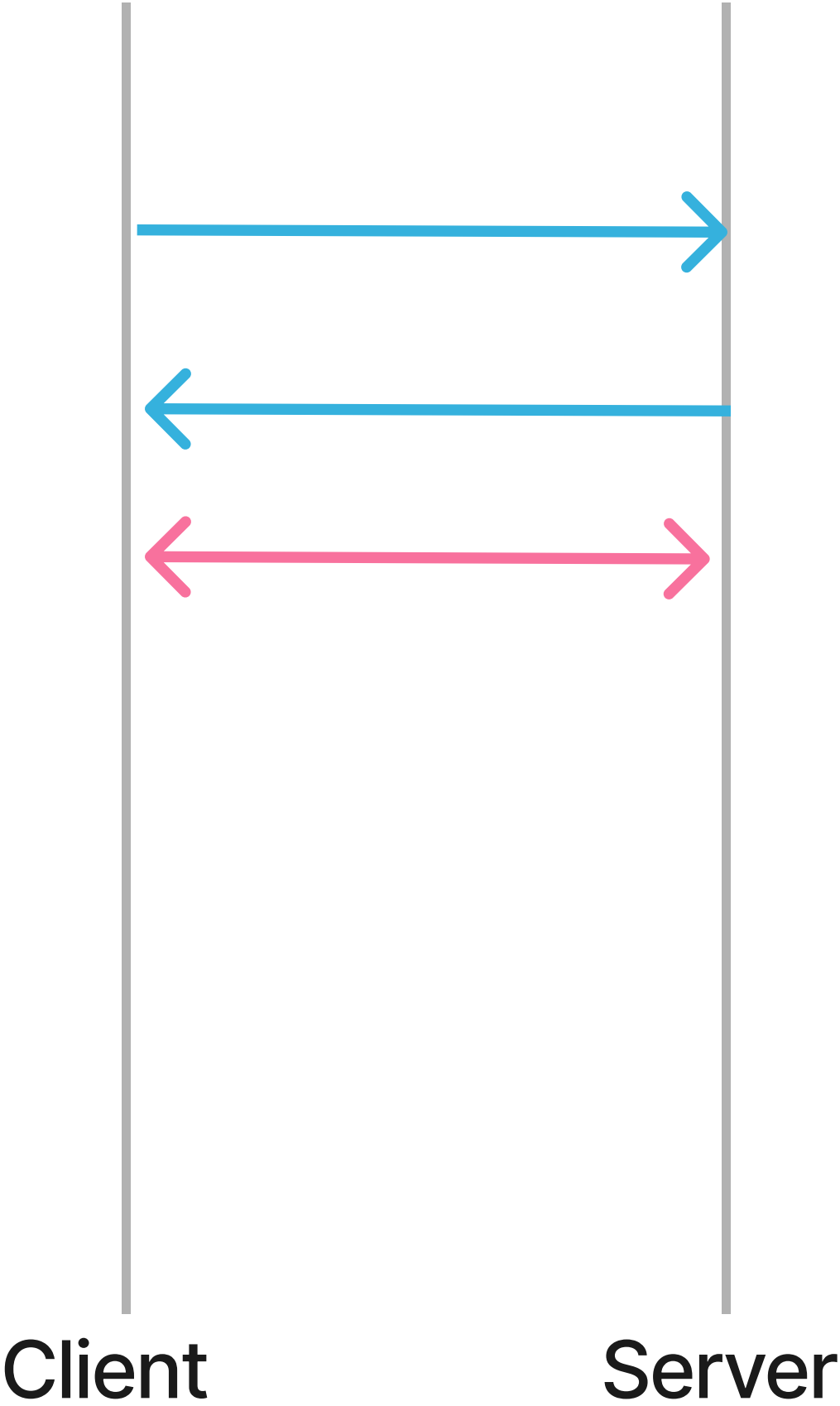
Long polling



SSE



WebSocket



Long polling vs Web Socket vs SSE

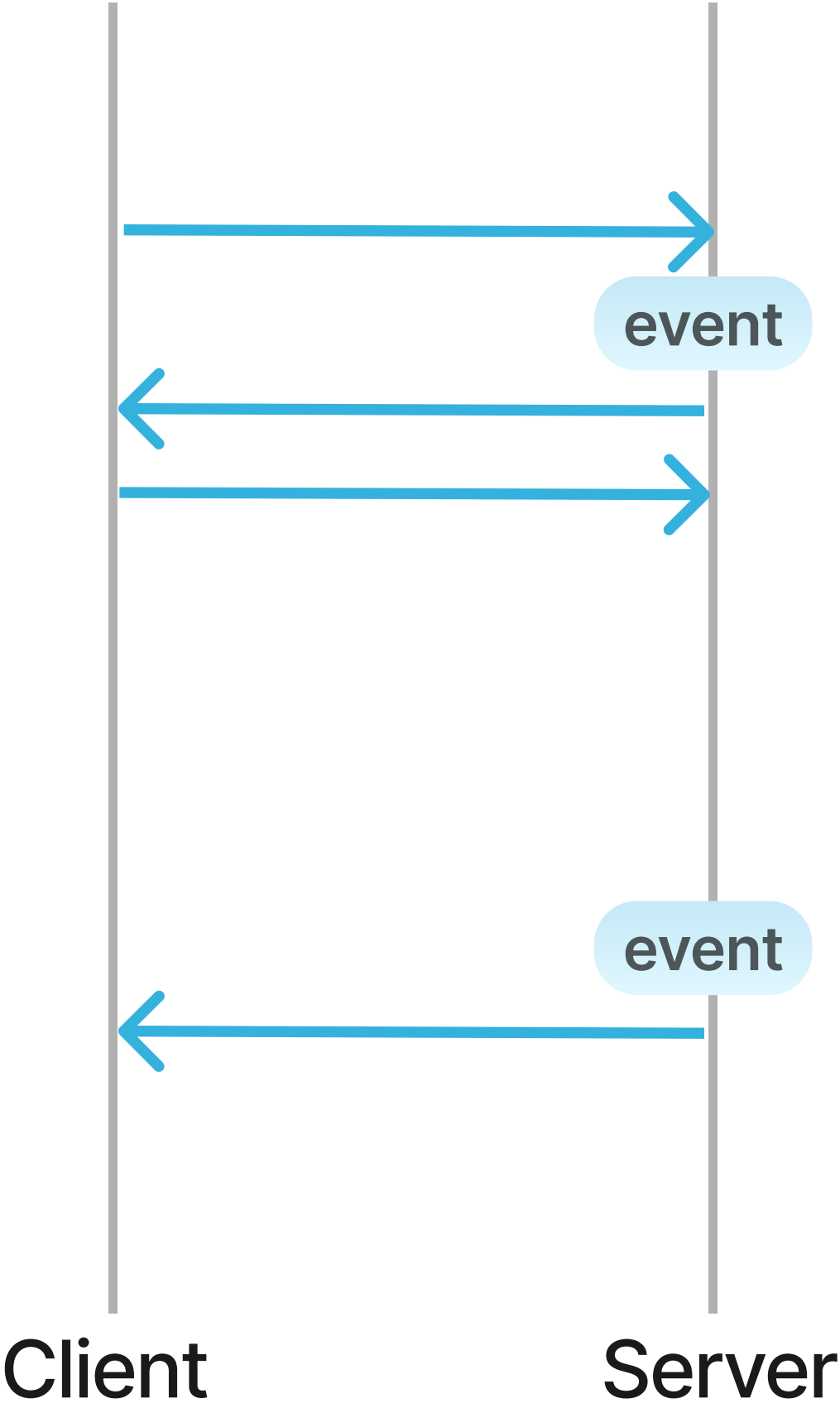
protocol

http

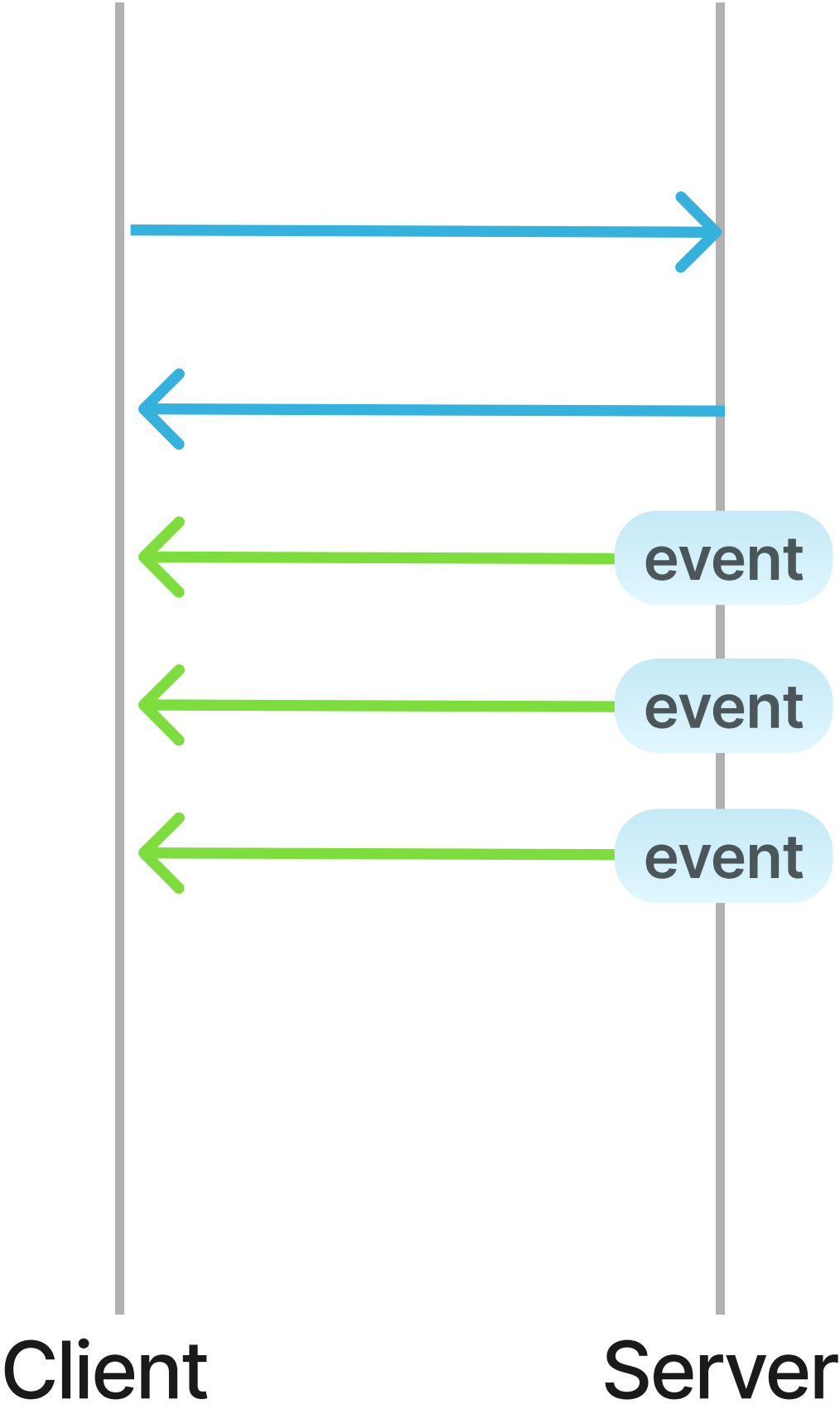
EventSource

WebSocket

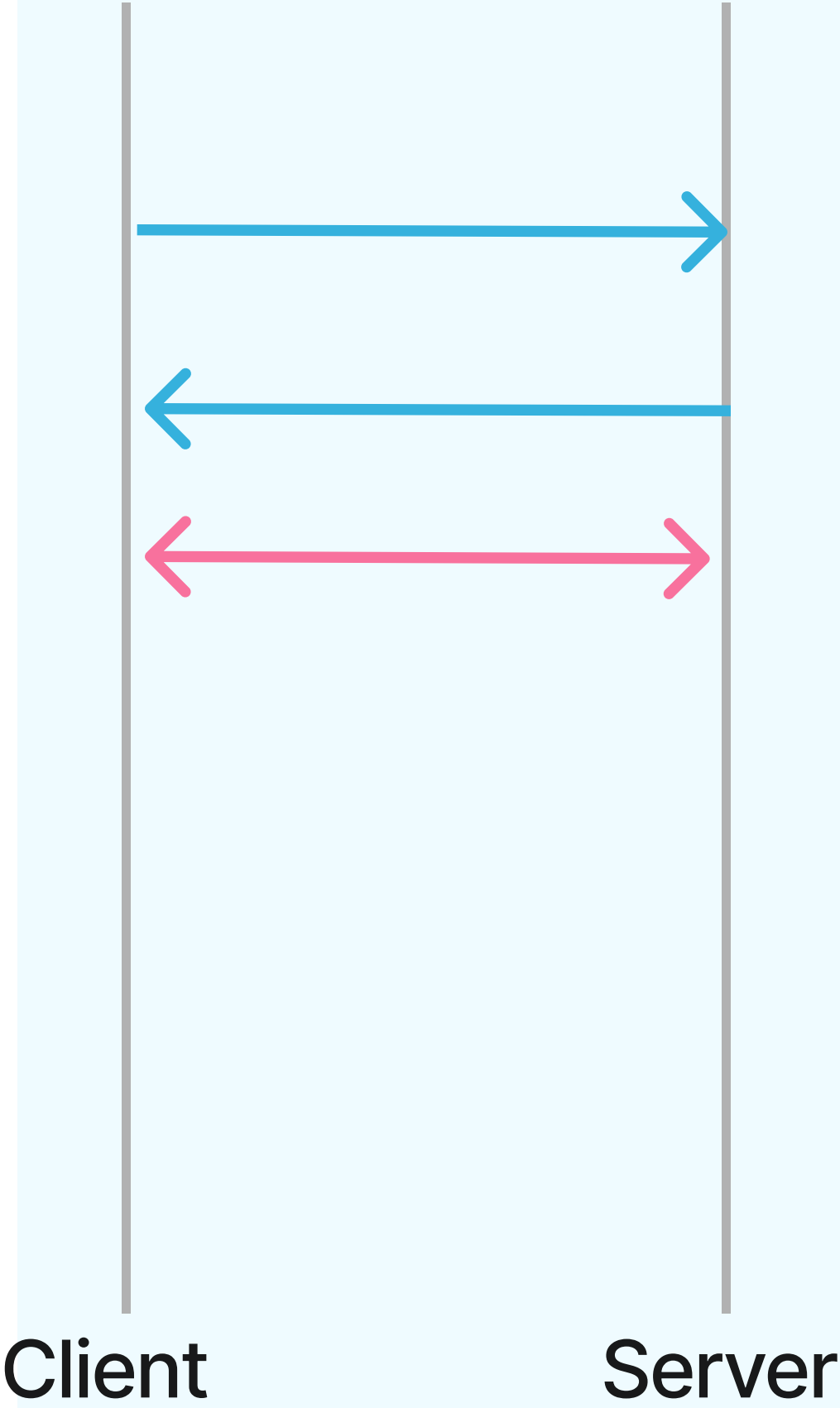
Long polling



SSE



WebSocket



SSE 선택 이유

SSE

- ✓ 최대 동시 접속 수 (브라우저 당)
 - HTTP 6개
 - HTTP2 100개
- ✓ 텍스트 기반 (event-stream)

서버의 상태 변화나 업데이트를 클라이언트가 수신만 하는 용도
ex) 뉴스 알람 기능, 타임라인 자동 갱신

WebSocket

- ✓ 최대 동시 접속 수 제한 X
- ✓ 바이너리 또는 텍스트 (프레임 기반)

여러 사용자가 동시에 같은 데이터를 수정하고, 그 결과를 실시간으로 반영해야 하는 상황
ex) Notion, Google Docs, 실시간 채팅

SSE 선택 이유

SSE

- ✓ 최대 동시 접속 수 (브라우저 당)
 - HTTP 6개
 - HTTP2 100개
- ✓ 텍스트 기반 (event-stream)

➡ HTTP만으로 작동하고,설정이 단순하며,텍스트 전용이라 리소스/구현/네트워크 부담이 적다

서버의 상태 변화나 업데이트를 클라이언트가 수신만 하는 용도
ex) 뉴스 알람 기능, 타임라인 자동 갱신

WebSocket

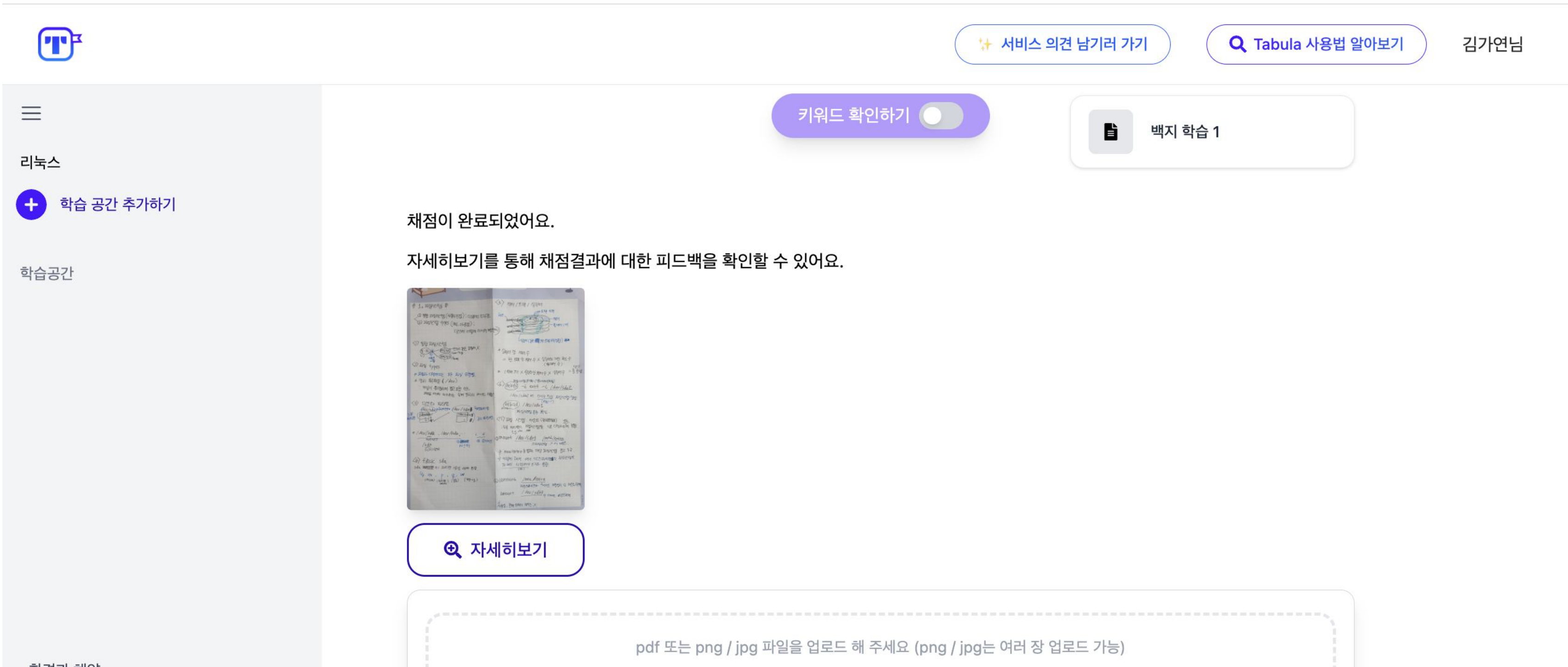
- ✓ 최대 동시 접속 수 제한 X
- ✓ 바이너리 또는 텍스트 (프레임 기반)

여러 사용자가 동시에 같은 데이터를 수정하고, 그 결과를 실시간으로 반영해야 하는 상황
ex) Notion, Google Docs, 실시간 채팅

Server-sent event

사용

- 1. 학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답받은 워크스페이스 id로 리다이렉트
- 2. 백지학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답값 워크스페이스 리스트 상태에 추가



Server-sent event

사용

1. 학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답받은 워크스페이스 id로 리다이렉트
2. 백지학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답값 워크스페이스 리스트 상태에 추가

중복되는 부분

EventSource 객체 생성, 이벤트 핸들러 등록, 데이터 파싱, SSE 연결 해제

다른 부분

api url, 완료되었을 때 로직

사용처

- 1. 학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답받은 워크스페이스 id로 리다이렉트
- 2. 백지학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답값 워크스페이스 리스트 상태에 추가

어떻게 여러 곳에서 쉽게 SSE 스트리밍을 시작하고,
각 로직에 맞게 처리할 수 있을까?

Server-sent event

사용처

1. 학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답받은 워크스페이스 id로 리다이렉트
2. 백지학습자료 업로드 → SSE 스트리밍 시작 → progress 동안 퍼센트 받아 증가 → complete 시 응답값 워크스페이스 리스트 상태에 추가

어떻게 여러 곳에서 쉽게 SSE 스트리밍을 시작하고,

각 로직에 맞게 처리할 수 있을까?

react-query

```
return useMutation({
  mutationFn: ({ spaceId, spaceName }: UpdateWorkspace) =>
    updateWorkspaceName(spaceId, spaceName),
  onError: (error) => {
    addToast('워크스페이스 이름 변경에 실패했습니다.', 3, 'error');
    console.error('Error updating workspace name:', error);
  },
  onSuccess: () => {
    addToast('워크스페이스 이름이 변경되었습니다.', 3, 'default');
    queryClient.invalidateQueries({ queryKey: ['workspaceList'] });
  },
});
```

useSSE (url , onSuccess,onProgress,onError)

connnect(url 이벤트 객체 생성)

- progress : 진행 중 데이터(예: 중간 상태, 부분 결과) 수신
event.data JSON 파싱, onError(), onProgress() 콜백 실행
- complete : 최종 결과 수신
event.data JSON 파싱, onSuccess() 콜백 실행
- onerror : 브라우저 자체 오류 이벤트, 연결 문제 등 네트워크 오류 처리

구현 코드

```
useSSE<ResponseType, ProgressData>({
  url,
  onSuccess: (response) => {
    const spaceId = response.spaceId;
    queryClient.invalidateQueries({ queryKey: ['workspaceList'] });
    router.push(`./${spaceId}`);
  },
  onError: (error) => {
    console.error(
      error.message ? error.message : 'An error occurred during SSE',
    );
  },
  onProgress: (response) => {
    let progress = 0;
    if (typeof response === 'number') {
      progress = response;
    } else if (typeof response.progress === 'number') {
      if (progress > 0) {
        setPercent(progress);
      }
      setPercent(progress);
    }
  },
});
```

```
useSSE<ResultResponseType, ProgressData>({
  url,
  onSuccess: (response) => {
    completeLoadingResult({
      resultId: response.resultId,
      resultFileName: '',
      resultImages: response.results || [],
    });
  },
  onError: (error) => {
    console.error('SSE Error:', error);
  },
  onProgress: (response) => {
    let progress = 0;
    if (typeof response === 'number') {
      progress = response;
    }
    setPercent(progress);
  },
});
```